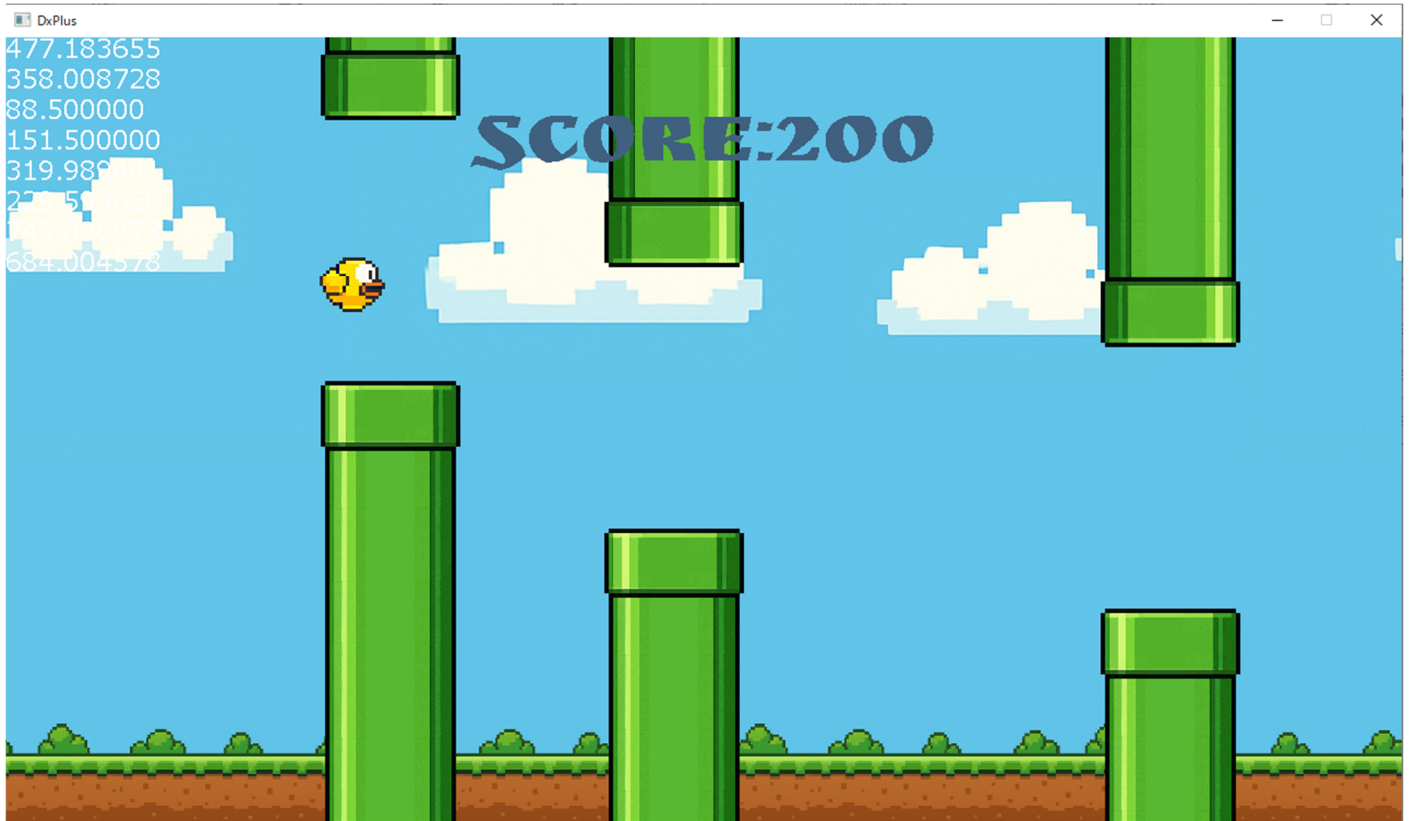


2D ゲームプログラミング I – 10 フラッピーバード① [1]

今回の概要

今回から2回に分けて、横スクロール型のアクションゲーム「フラッピーバード」を制作します。



【今回の目標】

- ・ 背景の描画と無限スクロール処理
- ・ プレイヤーのジャンプ操作
- ・ プレイヤーのアニメーション処理

【次回予定】

- ・ 土管の出現処理
- ・ 円と矩形によるあたり判定

2D ゲームプログラミング I – 10 フラッピーバード① [2]

目次

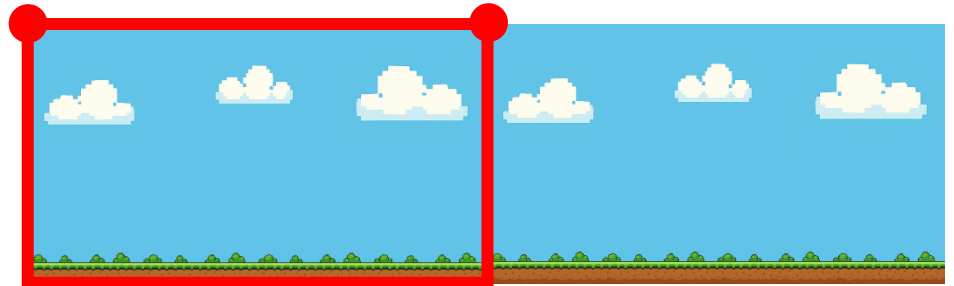
- 01) Entity2D 構造体の作成
- 02) 背景の準備
- 03) 背景の初期設定
- 04) 背景の描画
- 05) 背景のスクロール
- 06) プレイヤーの準備
- 07) プレイヤーの初期設定
- 08) プレイヤーの描画
- 09) プレイヤーの落下
- 10) プレイヤーのジャンプ
- 11) アニメーション定義の準備
- 12) アニメーション用変数の初期設定
- 13) プレイヤーのアニメーション

2D ゲームプログラミング I - 10 フラッピーバード① [3]

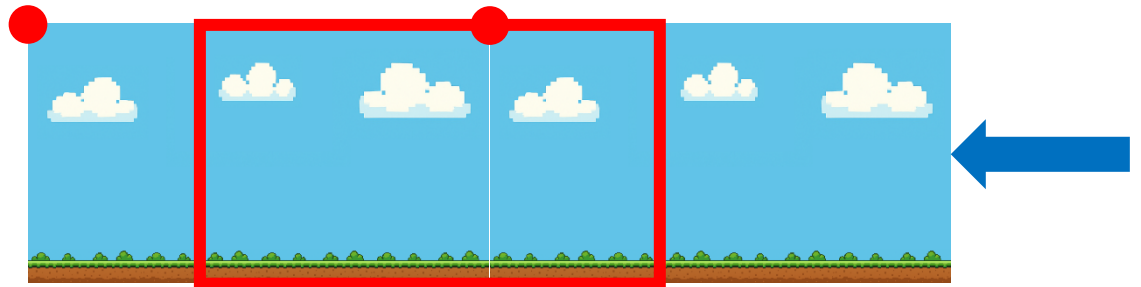
解説

03) 背景の初期設定

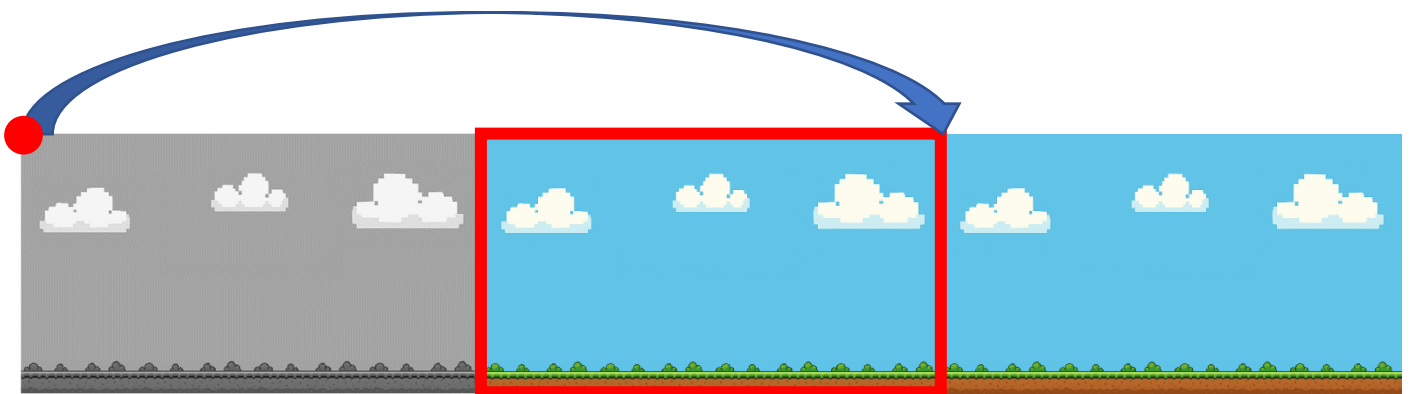
05) 背景のスクロール



赤枠が画面だとした場合、初期設定では上のように配置するとよい。



左に-SCROLL_SPEED の速度で移動させていく。



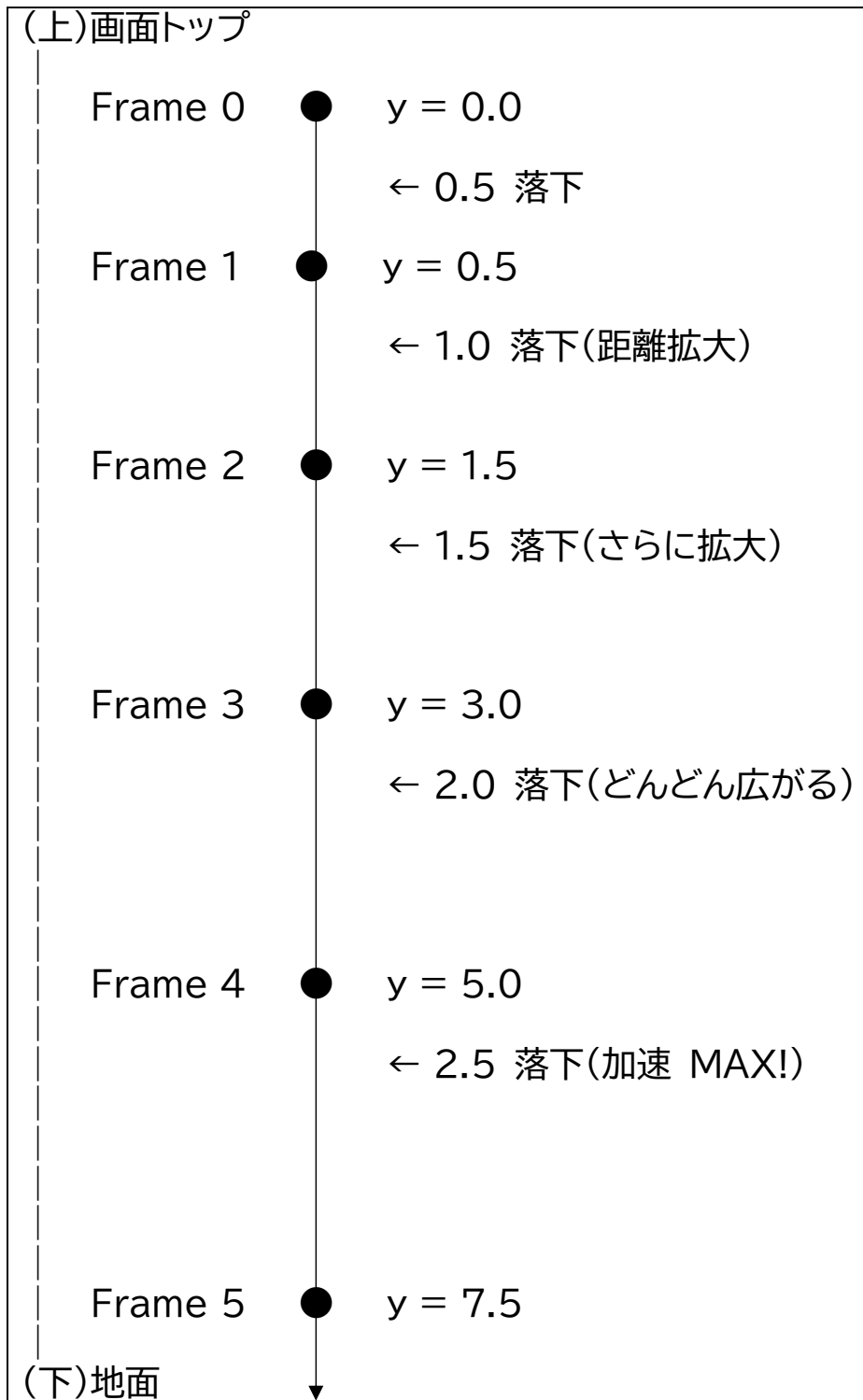
画面から完全に消えるのは、`backs[i].position.x` が `-DxPlus::ClientWidth` 以下になった瞬間ですね。

その際に `backs[i].position.x` に `DxPlus::ClientWidth * 2` を足すことで、無限スクロールが実現できるのです。

2D ゲームプログラミング I - 10 フラッピーバード① [4]

09) プレイヤーの落下

ボールを落としたときの動き



左の図は、ボールを真下に落としたときの動きを表しています。

物体が落下する時、その動きは「下向きの等加速度直線運動」となります。

これは、時間が経つほど速度がどんどん増えていく運動で、物理では $v = g \times t$ という式で表されます。

つまり、「落ち始めはゆっくり、でも時間が経つと一気に加速していく」というのが落下の特徴です。

ゲームの中でも同じように、毎フレーム `velocity.y` に `GRAVITY(= 0.5f)` を加算していくことで、この「加速しながら落ちる動き」を自然に表現できます。

2D ゲームプログラミング I – 10 フラッピーバード① [5]

13) プレイヤーのアニメーション

スペースキーを押してジャンプした瞬間にアニメーションがスタート。



0 コマ目

4 フレームで次へ



1 コマ目

12 フレームで次へ



0 コマ目

もうずっとこのまま



player.velocity.y が 正なら(プレイヤーが落下中であれば)
player.spriteID にこの画像(2 コマ目)を設定すればよい。

2D ゲームプログラミング I – 10 フラッピーバード① [6]

評価基準

- C ランク(60～69 点): 背景のスクロールまで実装できている。(TODO:05)
- B ランク(70～79 点): プレイヤーのジャンプまで実装できている。(TODO:10)
- A ランク(80～89 点): プレイヤーのアニメーションまで実装できている。(TODO:13)
- S ランク(90 点以上): ひとまとまりの処理を関数に分けられている。
例)
- ・ 背景をスクロールさせる処理
 - ・ プレイヤーがジャンプする処理
 - ・ プレイヤーのアニメーションを進める処理
 - ・ プレイヤーが落下する処理
 - ・ 背景を描画する処理
 - ・ プレイヤーを描画する処理

このように「意味のまとまり」で処理を分けていくことは、
後でコードを見直したり、修正したり、機能を追加したりするときにとっても役立ちます。

今回はゲームとして最低限動けば OK ですが、
S ランクを目指す人は「読みやすさ」「整理整頓」にも挑戦してみましょう！